

A1 · Answer Key and Demo Script

CS425 · Speaking the Application Layer

INSTRUCTOR COPY

Graded pass/fail. Every transcript below is real output captured from `onix.boisestate.edu` while this was written, so you can run any of it at the podium and get the same shape. Numbers will differ; the shapes will not.

WHAT IS ON THE BOX

THING	MEASURED ON ONYX
OS	Red Hat Enterprise Linux 9.8 (Plow), kernel 5.14
dig	DiG 9.16.23-RH
curl	7.76.1, OpenSSL 3.5.5, nghttp2
nc	nmap-ncat 7.92 at <code>/usr/bin/ncat</code> , not BSD netcat
ping	iputils 20210202
resolvers	10.50.18.111 and 10.50.18.112
outbound 80 / 443	open
outbound 53 elsewhere	blocked , which is why two dig switches fail
outbound ICMP	blocked , which is the whole of Round 6
<code>~/bashrc</code>	no PS1 guard; <code>~/bash_profile</code> sources it

ROUND 2 · DIG

Reading the answer
<pre>\$ dig boisestate.edu ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 59539 ;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1 boisestate.edu. 300 IN A 132.178.96.59 ;; Query time: 1 msec ;; SERVER: 10.50.18.111#53(10.50.18.111)</pre> SAY: four things to point at. STATUS is the verdict. ANSWER: 1 is how many records came back. 300 is the TTL. SERVER is who answered, and it is a campus box, not the internet.

The TTL, and why the original version of this exercise was wrong
<pre>\$ dig +noall +answer boisestate.edu # five times boisestate.edu. 300 IN A 132.178.96.59 ← 300 every single time \$ dig +noall +answer example.com ; sleep 3 ; dig +noall +answer example.com example.com. 218 IN A 172.66.147.243 example.com. 241 IN A 172.66.147.243 ← it went UP</pre> SAY: <code>boisestate.edu</code> never moves because the campus resolver is AUTHORITATIVE for that zone: it is not caching the answer, it <i>is</i> the answer. <code>example.com</code> moves because it really is cached, and it sometimes moves UP because <code>resolv.conf</code> lists two resolvers and you are getting whichever answers first, with caches of different ages. A group that notices the increase and is bothered by it has understood caching better than one that just wrote "it counts down".

The two switches that fail, and the one that lies

```
$ dig @1.1.1.1 boisestate.edu
;; connection timed out; no servers could be reached

$ dig +trace boisestate.edu
;; Received 239 bytes from 10.50.18.111#53(10.50.18.111) in 19 ms ← and then it stops

$ dig nosuchthing.boisestate.edu
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 8848
;; flags: qr rd ra; QUERY: 1, ANSWER: 0

$ dig +short nosuchthing.boisestate.edu
(nothing at all, and it exits 0)
```

SAY: the first two fail for **THE SAME REASON:** both need to send a query directly to a DNS server out on the internet, and outbound port 53 is blocked. That is a middlebox making a policy decision, and in A2 they meet one that answers *wrongly* instead of not at all, which is worse. **THEN THE TRAP:** +short prints an empty line for a name that does not exist. The empty line is a real answer, badly displayed. This is A2 station 4, exactly.

The nuance worth having ready

Someone will run `dig +short boisestate.edu AAAA` and get nothing, and conclude the name does not exist. It does. The full output says **status: NOERROR with ANSWER: 0: the name exists, that record type does not**. Two different causes, one identical empty line, and only the long form tells them apart. If a group finds this on their own, put it on the board.

PASS BAR: the status field, answer and TTL are written down, and the group noticed that +short printed nothing for the bad name.

ROUND 3 · SSH KEYS AND MULTIPLEXING

Cold versus warm, measured from a laptop

```
$ ssh -O exit onyx ; time ssh onyx true
cold (full handshake): 1105 ms

$ time ssh onyx true
warm (reused master): 248 ms

$ ping -c 3 onyx.boisestate.edu
rtt min/avg/max = 31.4/33.7/35.3 ms
```

SAY: about 1.1 seconds against a 31 ms round trip. Ask what the extra time bought: a TCP handshake, a version exchange, a key exchange, and authentication. Accept any estimate in the range of a handful to a few dozen round trips. **DO NOT LET THEM CONCLUDE IT IS ALL NETWORK TIME:** key exchange is arithmetic, and on a fast link the crypto dominates. The warm case is roughly 4x faster and still not zero, because starting a process on the far end is not free.

PASS BAR: everyone logs in without a password, and two real timings are recorded.

ROUND 4 · CURL

Both halves of the conversation

```
$ curl -v http://example.com/
> GET / HTTP/1.1
> Host: example.com
< HTTP/1.1 200 OK
< Content-Type: text/html
```

SAY: > is what you sent, < is what came back. Four lines of text is the whole of an HTTP exchange. They will write this by hand in Round 5.

Where the time goes · this is the layer model, printed

```
$ curl -o /dev/null -s -w 'dns=%{time_namelookup} connect=%{time_connect} tls=%{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total} code=%{http_code}\n' https://www.boisestate.edu/
dns=0.002637 connect=0.020129 tls=0.055797 ttfb=0.088997 total=0.144840 code=200
```

```
$ ... the same thing against http:// instead
dns=0.022634 connect=0.034200 tls=0.000000 ttfb=0.057717 total=0.057802 code=200
```

SAY: the answers to the three questions are all in these two lines. (1) Largest jump is usually connect→tls or tls→ttfb. (2) Between connect and ttfb sits the TLS handshake and then the server thinking about your request. (3) tls collapses to **EXACTLY ZERO** over plain HTTP, because there is no TLS handshake to measure. Cumulative, not per-phase: each number is time since the start. Subtract neighbors to get the cost of a phase.

PASS BAR: all six numbers recorded, and the group said tls is the one that went to zero.

ROUND 5 · NC

Refused versus timed out, which is A2 in miniature

```
$ nc -vz onyx.boisestate.edu 22
Ncat: Connected to 132.178.227.11:22.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
```

```
$ nc -vz onyx.boisestate.edu 9
Ncat: Connection refused.
```

```
$ nc -vz -w 3 example.com 8081
Ncat: Connection to 172.66.147.243 failed: TIMEOUT.
```

SAY: three outcomes, three meanings. Connected, refused (something answered "no"), and timeout (nothing answered at all). Those are A2 stations 1, 3 and 2 respectively, a week early and on a machine nobody deliberately broke.

HTTP by hand, and why the sleep is there

```
$ printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' | nc -v example.com 80
Ncat: Connected to 172.66.147.243:80.
Ncat: 56 bytes sent, 0 bytes received in 0.06 seconds.      ← no reply!
```

```
$ { printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n'; sleep 3; } | nc example.com 80
HTTP/1.1 200 OK
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
```

SAY: nmap-ncat closes the connection the instant stdin runs out. The request left the machine, and nc hung up before the answer arrived. **SHOW THEM THE "56 BYTES SENT, 0 BYTES RECEIVED" LINE:** the tool told them exactly what happened, in numbers, and it is the sort of line people scroll past. The \r\n question is the same rule SMTP uses, so this is P1 preparation. A server given bare \n may tolerate it or may hang; either way they cannot see the difference in a terminal, which is the lesson.

PASS BAR: an HTTP status line obtained without a browser, and the group can say what the two nc failure messages mean.

ROUND 6 · PING, AND THE CONTRADICTION

The best ninety seconds in the activity

```
$ ping -c 3 onyx.boisestate.edu
3 packets transmitted, 3 received, 0% packet loss
```

```
$ ping -c 3 example.com
3 packets transmitted, 0 received, 100% packet loss
```

... and in Round 4, twenty minutes earlier, from this same machine:

```
$ curl -o /dev/null -s -w 'code=%{http_code}\n' http://example.com/
code=200
```

SAY: outbound ICMP is blocked from Onyx, so every host on the internet looks dead to ping from here, including one that served them a web page in the same period. Put both lines on the board together.

Answers: (1) ICMP is network layer, riding directly in IP, so it has no ports and no idea what a service is. (2) No: pinging a host says nothing about any program running on it. (3) A failed ping is consistent with a dead host, a filtered ICMP path, a filter at either end, or a middlebox in between. It rules almost nothing out, which is why "it doesn't ping" is not a diagnosis.

PASS BAR: both ping results recorded side by side, and the group names more than one cause for the failure.

ROUND 7 · MAKING IT STICK

The function, loaded and proven

```
$ type probe
probe is a function
```

```
$ probe example.com 80
connect=0.033370 total=0.057519 code=200
```

```
$ ssh onyx probe example.com 80      # non-interactive, from the laptop
connect=0.033370 total=0.057519 code=200
```

SAY: the third one is the one that matters. That shell had no prompt and no terminal, and the function was still there, because bash reads `~/.bashrc` when a remote shell daemon starts it and Onyx has no guard line stopping it. **VERIFIED ON ONYX**, so if a student's fails, the cause is in their file, not in the machine.

FUNCTION VERSUS ALIAS: an alias is textual substitution at the front of a command, so there is no way to drop \$1 and \$2 into the middle of a URL. Positional parameters are the entire reason this has to be a function.

PASS BAR: probe survives a logout, and the group can say why an alias cannot do the job.

Running the room

- **Round 2 is the spine.** If time is short, cut Round 6's switch table but keep the `ping example.com` demonstration; it is ninety seconds and it is the best thing here.
- A broken `~/.bashrc` can cost a student their shell. `ssh onyx -t 'bash --norc'` gets them back in to fix it.
- The Round 5 listener must bind to `127.0.0.1` and must be killed. It is a shared department machine.
- Send them out with the worksheet graded and returned before A2, which is the following week and reuses all four commands.